CCNA Lab Workbook

What this is: the hands-on companion to the Study Guide — 17 labs that turn every major topic you read into something your fingers have done. The exam includes lab items with no multiple choice to lean on, and job interviews put you in front of a CLI. This book is how you get ready for both.

How to use it: each lab names the Study Guide sections it exercises — read those first, then build the lab **without looking at the solution**. Struggle a little; that's the point. The solution box is for when you're stuck or for checking your work afterwards. Every lab ends with a **Break it** challenge: deliberately damage the network and fix it, because troubleshooting broken networks is the actual job.

Platform: written for **EVE-NG** (IOL or vIOS images for routers and switches, **VPCS** for end hosts), and everything works the same in GNS3 or CML. Only the wireless GUI work points elsewhere — see the closing note after Lab 17. No server of your own? The **DevNet Lab Companion** (CCNA_DevNet_Labs.pdf) maps every lab onto Cisco's free cloud sandboxes.

The Labs

#	Lab	Guide chapters
0	EVE-NG setup & habits	—
1	First contact: IOS basics & SSH	6
2	Port security	6
3	VLANs & access ports	7
4	Trunks, DTP & the native VLAN	8
5	Inter-VLAN routing (two ways)	8
6	Spanning Tree	9
7	EtherChannel	10
8	Addressing design (VLSM)	11–12
9	Static, default & floating routes	15
10	OSPF single area	16
11	IPv6 & SLAAC	13
12	HSRP	14
13	IP services: DHCP, NTP, syslog, SNMP	17
14	NAT & PAT	17
15	ACLs	19
16	Layer 2 defenses	18
17	Capstone: build a company	everything

Lab 0 — EVE-NG Setup & Habits

No tasks to grade here — just the platform decisions that make the next 17 labs smooth.

Images. You need one Layer 3 image (a router) and one Layer 2 image (a switch). The usual choices in EVE-NG are **IOL** (`L3` and `L2` images — tiny, fast, boot in seconds) or **vIOS** (`vios` router + `viosl2` switch). Either works for every lab in this book. For end hosts use **VPCS** — it boots instantly and does exactly what a lab PC needs: `ip`, `ping`, `trace`, `dhcp`.

VPCS survival kit (type `?` for the rest):

```
VPCS> ip 192.168.10.20 255.255.255.0 192.168.10.1   ! address, mask, gateway
VPCS> ip dhcp                                       ! or get it from DHCP
VPCS> show ip                                       ! what do I have?
VPCS> ping 192.168.10.1
VPCS> trace 8.8.8.8
VPCS> save                                          ! keep it after reboot
```

Habits that pay off all book long:

1. **Save constantly.** `copy running-config startup-config` (or `wr`) on every device before you stop — EVE-NG nodes that get wiped or powered off lose the running config. `save` on every VPCS.
2. **Export finished labs.** EVE-NG can export all startup configs into the lab file (More → Export all CFGs) — do it when a lab works, and you can rebuild it any time.
3. **Name everything.** `hostname SW1` is the first command in every solution here — when four terminal tabs are open, `Switch>` four times is misery.
4. **Verify before you believe.** After every task, prove it worked with a `show` command. The habit of checking rather than assuming is the most CCNA-transferable skill in this book.

Lab 1 — First Contact: IOS Basics & SSH

Guide sections: 6.4–6.7 · **Time:** ~30 min

The unglamorous lab that everything else stands on: get into a device, move between modes, configure remote access properly, and save.

Topology

Device	Image	Connects to
SW1	L2 switch	PC1 on e0/1
PC1	VPCS	SW1 e0/1

Addressing

Device	Interface	IP
SW1	VLAN 1 (SVI)	192.168.1.2/24
PC1	eth0	192.168.1.10/24

Tasks

1. Explore before configuring: from user EXEC, enter privileged EXEC, then global config. At each level, type `?` and skim what lives there.
2. Set hostname **SW1**, an enable secret of `labpass`, and a login banner that would make a lawyer happy.
3. Give SW1 its management IP on the VLAN 1 SVI and a default gateway of 192.168.1.1 (it doesn't exist yet — set it anyway and explain to yourself why a Layer 2 switch even has one).

4. Configure SSH: domain name, RSA keys (2048-bit), a local user `admin` with secret `sshpass`, and vty lines that accept **only** SSH with local login.
5. Address PC1 and verify: ping SW1, then SSH to it from... wait, VPCS can't SSH. Verify instead with `show ip ssh` and `show ssh` on SW1, and ping both directions.
6. Save the config, reload SW1, and confirm the config survived.

Verification checklist

- `show running-config` shows the hostname, banner, SVI address
- `show ip ssh` reports SSH version 2 enabled
- `ping 192.168.1.10` from SW1 succeeds
- After `reload`: config intact (if not, you skipped the most important command)

Solution

```
Switch> enable
Switch# configure terminal
Switch(config)# hostname SW1
SW1(config)# enable secret labpass
SW1(config)# banner motd # Authorized access only #
SW1(config)# interface vlan 1
SW1(config-if)# ip address 192.168.1.2 255.255.255.0
SW1(config-if)# no shutdown
SW1(config-if)# exit
SW1(config)# ip default-gateway 192.168.1.1
SW1(config)# ip domain-name lab.local
SW1(config)# crypto key generate rsa modulus 2048
SW1(config)# ip ssh version 2
SW1(config)# username admin secret sshpass
SW1(config)# line vty 0 4
SW1(config-line)# transport input ssh
SW1(config-line)# login local
SW1(config-line)# end
SW1# copy running-config startup-config
```

PC1: `ip 192.168.1.10 255.255.255.0 192.168.1.1` then `save`.

Why the default gateway on a switch? SW1 forwards frames without it — but its own **management traffic** (your SSH session from another subnet) needs a way back out. The gateway is for the switch as a host, not as a switch.

Break it

Have a friend (or your future self) set `transport input none` on the vty lines. Symptom: SSH connects are refused but pings work. Find it with `show running-config | section line vty`.

Lab 2 — Port Security

Guide sections: 6.8 · **Time:** ~30 min

Topology

Same as Lab 1, plus **PC2** (VPCS) that you'll treat as the intruder's laptop. Connect PC1 to SW1 e0/1; leave PC2 disconnected for now.

Tasks

1. On e0/1: make it a static access port and enable port security with a maximum of **1** MAC address, **sticky** learning, and the default violation mode.
2. Generate traffic from PC1, then confirm its MAC is now written into the running config (that's what sticky means).
3. Now be the attacker: disconnect PC1, connect PC2 to e0/1, and send traffic. Watch what happens to the port.
4. Recover the port properly and reconnect PC1.
5. In one sentence each, explain when you'd choose violation mode `shutdown` vs `restrict`. (18.3 has opinions.)

Verification checklist

- `show port-security interface e0/1` — port secure-up, violation mode, count
- After the attack: port is **err-disabled**, violation counter incremented
- `show running-config interface e0/1` shows the sticky MAC of PC1

Solution

```
SW1(config)# interface e0/1
SW1(config-if)# switchport mode access
SW1(config-if)# switchport port-security
SW1(config-if)# switchport port-security maximum 1
SW1(config-if)# switchport port-security mac-address sticky
```

Recovery after err-disable:

```
SW1(config)# interface e0/1
SW1(config-if)# shutdown
SW1(config-if)# no shutdown
```

If PC2's MAC got stickied during the attack, remove it first: `no switchport port-security mac-address sticky <mac>` — otherwise the port re-dies the moment PC1 talks. `shutdown` mode is loud and strict (a security event someone must investigate); `restrict` quietly drops the intruder while keeping the port up — friendlier for ports where a violation is more likely a

> mistake than an attack.

Break it

Set `maximum 1` on a port, then plug in an IP phone with a PC behind it (in EVE-NG: another switch with two VPCS). Watch the second MAC kill the port — this is the classic real-world port-security incident.

Lab 3 — VLANs & Access Ports

Guide sections: 7.1–7.5 · **Time:** ~40 min · **Diagram:** the VLANs figure in 7.1

Topology

Device	Connects to
SW1 (L2)	PC1 e0/1, PC2 e0/2, PC3 e0/3, PC4 e0/0... see below
PC1, PC2	VPCS — will be VLAN 10 (Staff)
PC3, PC4	VPCS — will be VLAN 20 (Students)

Addressing

Host	IP	VLAN
PC1	192.168.10.11/24	10
PC2	192.168.10.12/24	10
PC3	192.168.20.13/24	20
PC4	192.168.20.14/24	20

Tasks

1. Before any VLAN config: address all four PCs and confirm PC1 can ping PC3 even though they're on different subnets... can it? Think about why the answer is no already — then explain what a ping from PC1 to PC3's address actually does on the wire. (Hint: ARP, gateway, 11.6.)
2. Create VLAN 10 named **STAFF** and VLAN 20 named **STUDENTS**.
3. Assign e0/1–2 to VLAN 10 and e0/3, e1/0 to VLAN 20 as static access ports.
4. Prove the walls exist: PC1↔PC2 pings succeed, PC3↔PC4 succeed, PC1→PC3 fails.
5. Look at `show vlan brief` and find every port the default VLAN still owns.

Verification checklist

- `show vlan brief` — VLANs 10/20 with correct names and ports
- `show interfaces e0/1 switchport` — operational mode: static access, VLAN 10
- The ping matrix from task 4

Solution

```
SW1(config)# vlan 10
SW1(config-vlan)# name STAFF
SW1(config-vlan)# vlan 20
SW1(config-vlan)# name STUDENTS
SW1(config-vlan)# exit
SW1(config)# interface range e0/1 - 2
SW1(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 10
SW1(config-if-range)# interface e0/3
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 20
SW1(config-if)# interface e1/0
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 20
```

Task 1's answer: even with no VLANs, PC1 never ARPs for PC3 — different subnet means PC1 sends to its **gateway**, which doesn't exist, so the ping dies at the ARP step. VLANs then make the separation physical as well as logical.

Break it

Move PC2's port to VLAN 20 but leave its IP in the 192.168.10.0 subnet. Now nothing works for PC2 — practice diagnosing "right cable, wrong VLAN vs right VLAN, wrong IP" from the outside using only pings and `show` commands.

Lab 4 — Trunks, DTP & the Native VLAN

Guide sections: 8.1–8.3 · **Time:** ~40 min

Topology

Two switches now. Keep Lab 3's SW1; add SW2 with PC5 and PC6.

Link	Purpose
SW1 e2/0 ↔ SW2 e2/0	the trunk
SW2 e0/1 → PC5	VLAN 10, 192.168.10.15/24
SW2 e0/2 → PC6	VLAN 20, 192.168.20.16/24

Tasks

1. Recreate VLANs 10 and 20 on SW2 (no VTP — type them; ponder 8.3's warning about why this book won't teach you to lean on VTP).
2. Configure the e2/0 link as a **manual** 802.1Q trunk on both ends — mode trunk, DTP negotiation off (`nonegotiate`).
3. Change the native VLAN on the trunk to **99** on both sides, and create VLAN 99 named **NATIVE-UNUSED** with no access ports in it.
4. Restrict the trunk to carry only VLANs 10, 20 and 99.
5. Prove it: PC1↔PC5 (VLAN 10 across the trunk) and PC3/PC4↔PC6 (VLAN 20 across the trunk), and PC1→PC6 still fails.
6. Deliberately set the native VLAN to 99 on SW1 only, leaving SW2 at 1. Read the CDP error messages that appear, then fix it.

Verification checklist

- `show interfaces trunk` — mode on, native VLAN 99, allowed 10,20,99
- `show interfaces e2/0 switchport` — negotiation of trunking: off
- The cross-switch ping matrix

Solution

Both switches, symmetric:

```
SW1(config)# vlan 99
SW1(config-vlan)# name NATIVE-UNUSED
SW1(config-vlan)# exit
SW1(config)# interface e2/0
SW1(config-if)# switchport trunk encapsulation dot1q   ! IOL/vIOS need this
SW1(config-if)# switchport mode trunk
SW1(config-if)# switchport nonegotiate
SW1(config-if)# switchport trunk native vlan 99
SW1(config-if)# switchport trunk allowed vlan 10,20,99
```

The task 6 mismatch produces `%CDP-4-NATIVE_VLAN_MISMATCH` — worth seeing once on purpose so you recognize it instantly when it's not on purpose. Traffic on the mismatched native VLANs silently leaks between VLANs 1 and 99: exactly the hopping risk 8.2 warned about.

Break it

On SW2 only, remove VLAN 10 from the trunk's allowed list. VLAN 10 dies across the trunk; VLAN 20 keeps working. `show interfaces trunk` on **both** ends is the tool — the allowed lists must agree.

Lab 5 — Inter-VLAN Routing, Two Ways

Guide sections: 8.4 · **Time:** ~45 min · **Diagram:** the router-on-a-stick figure in 8.4

Part A — Router-on-a-stick

Add router **R1** to Lab 4's topology: R1 g0/0 ↔ SW1 e3/0.

1. Make SW1 e3/0 a trunk carrying VLANs 10 and 20.
2. On R1, create subinterfaces g0/0.10 and g0/0.20 with 802.1Q encapsulation and addresses 192.168.10.1/24 and 192.168.20.1/24.
3. Point every PC's gateway at its VLAN's .1 (VPCS: re-run `ip` with the gateway argument).
4. The moment of truth: PC1 → PC6. Trace it (`trace`) and narrate each hop out loud using 8.4's walkthrough — up tagged 10, routed, down tagged 20.

Part B — SVIs on a Layer 3 switch (the modern way)

1. Shut R1's g0/0 down. The network breaks — good.
2. On SW1: enable routing (`ip routing`), create SVIs for VLAN 10 and VLAN 20 with the same .1 addresses, and bring them up.
3. Same ping test. Same result, no router, no shared trunk bottleneck — 8.4's argument made physical.

Verification checklist

- Part A: `show vlans` on R1 shows both subinterfaces tagging correctly
- Part B: `show ip route` **on the switch** shows two connected subnets
- PC1↔PC6 works in both parts

Solution

Part A, R1:

```
R1(config)# interface g0/0
R1(config-if)# no shutdown
R1(config-if)# interface g0/0.10
R1(config-subif)# encapsulation dot1q 10
R1(config-subif)# ip address 192.168.10.1 255.255.255.0
R1(config-subif)# interface g0/0.20
R1(config-subif)# encapsulation dot1q 20
R1(config-subif)# ip address 192.168.20.1 255.255.255.0
```

Part B, SW1:

```
SW1(config)# ip routing
SW1(config)# interface vlan 10
SW1(config-if)# ip address 192.168.10.1 255.255.255.0
SW1(config-if)# no shutdown
SW1(config-if)# interface vlan 20
SW1(config-if)# ip address 192.168.20.1 255.255.255.0
SW1(config-if)# no shutdown
```

(Remove or shut the R1 subinterfaces first so the same addresses aren't alive twice — duplicate-address chaos is educational exactly once.)

Break it

In Part A, set g0/0.20's encapsulation to `dot1q 30` but leave the IP correct. VLAN 10 works, VLAN 20 doesn't, and nothing is "down." This is among the most instructive faults in the book — the config looks right at a glance.

Lab 6 — Spanning Tree

Guide sections: 9.1–9.9 · **Time:** ~50 min · **Diagram:** the STP triangle in 9.3

Topology

Three L2 switches in a triangle — the classic:

Link	Interfaces
SW1 ↔ SW2	e2/0 both ends
SW1 ↔ SW3	e2/1 both ends
SW2 ↔ SW3	e2/1 both ends

PC1 on SW2 e0/1, PC2 on SW3 e0/1, same subnet (192.168.1.11/24 and .12/24).

Tasks

1. Cable it up with default configs and nothing breaks — before reading on, say precisely why not (9.2), then find the proof: which port is blocking?
2. Map the tree: from `show spanning-tree` on all three switches identify the root bridge, each root port, each designated port, and the blocked port. Draw it on paper — yes, actually draw it.
3. Explain why that switch won root (9.3's tiebreakers), then force **SW1** to be root for VLAN 1 the best-practice way (9.7).
4. Watch a failover: start a continuous ping PC1→PC2, then shut the link the traffic is using. How many pings die before the blocked port takes over?

5. Configure PC-facing ports as edge ports: PortFast + BPDU Guard on SW2 e0/1 and SW3 e0/1.
6. Prove BPDU Guard works: move SW3's e2/1 cable to SW2's e0/1 (a "helpful" user plugging a switch into a wall port). Enjoy the err-disable.

Verification checklist

- `show spanning-tree` — root ID matches SW1 after task 3; port roles match your paper drawing
- Task 4: some pings lost, then recovery without your help
- `show spanning-tree interface e0/1 portfast` — enabled
- Task 6: `%SPANTREE-2-BLOCK_BPDUGUARD` and an err-disabled port

Solution

```
SW1(config)# spanning-tree vlan 1 root primary
SW2(config)# interface e0/1
SW2(config-if)# spanning-tree portfast
SW2(config-if)# spanning-tree bpduguard enable
```

(Same on SW3 e0/1.) Convergence in task 4 depends on the mode: with default Rapid PVST+ on modern images it's a blink (1–2 lost pings); if your image runs legacy PVST+ you'll sit through listening+learning — ~30 seconds of dead air, which is 9.11's whole argument for RSTP, experienced personally.

Break it

`spanning-tree vlan 1 priority 0` on SW3 — the "new switch takes over as root" disaster from 9.9. Watch the tree re-elect and traffic paths silently change (`show spanning-tree` before/after). Then protect against it: `root guard` on the proper SW1/SW2 ports, repeat, and watch it get blocked instead.

Lab 7 — EtherChannel

Guide sections: 10.1–10.4 · **Time:** ~30 min

Topology

SW1 and SW2, connected by **three** parallel links: e2/1↔e2/1, e2/2↔e2/2, e2/3↔e2/3. A PC on each switch, same subnet.

Tasks

1. First, bring all three links up **without** EtherChannel and check `show spanning-tree` — how many of the three does STP actually use? This is 10.1's "you paid for three, you use one" in the flesh.
2. Bundle all three into Port-channel 1 using **LACP**, with SW1 active and SW2 passive.
3. Make the port-channel a trunk (both the physical members and the logical interface should agree — configure the trunk on the Po interface and watch it push down).
4. Re-check `show spanning-tree`: how many links does STP see now?
5. Read `show etherchannel load-balance` and explain (10.4) why a single file transfer between the two PCs will never exceed one member link's speed.

Verification checklist

- `show etherchannel summary` — Po1 with flags **SU**, members bundled `(P)`
- `show spanning-tree` — one Po1 interface where three links used to be
- PC↔PC pings survive shutting any one member link

Solution

```
SW1(config)# interface range e2/1 - 3
SW1(config-if-range)# channel-group 1 mode active
SW1(config)# interface port-channel 1
SW1(config-if)# switchport trunk encapsulation dot1q
SW1(config-if)# switchport mode trunk
```

SW2 identical but `channel-group 1 mode passive`. Active/passive forms (active/active also works; passive/passive never does — someone must speak first). One flow's frames all hash to one member (10.4): keeping a flow on one link is what guarantees its frames arrive in order.

Break it

Set e2/3 on one side to a different trunk native VLAN than its peers before bundling. LACP refuses to bundle it (`(s)` or suspended) — EtherChannel demands identical member configs, and `show etherchannel summary` plus the log tells you exactly which member is the misfit.

Lab 8 — Addressing Design (VLSM)

Guide sections: 11.1–12.8 · **Time:** ~45 min, mostly paper

Design first, configure second — the configuring is the reward.

The brief

You've been given **172.16.40.0/22** for a two-site company:

Network	Hosts needed
HQ staff	400
HQ servers	60
Branch staff	100
Branch guest Wi-Fi	40
HQ↔Branch router link	2

Tasks

1. On paper: carve the /22 with VLSM — biggest first (12.7). For each subnet write network address, mask, first/last usable host, broadcast. Check your math with the drill sheet's method, not a calculator.
2. Sanity checks before touching a device: does everything fit? How much space is left over? What prefix did the router link get, and why /30 (or /31 — 12.8) rather than anything bigger?
3. Build the skeleton in EVE-NG: R1 (HQ) ↔ R2 (Branch) on the point-to-point subnet, one VPCS per LAN subnet hanging off each router (one interface per subnet is fine — this lab is about addressing, not switching).
4. Address everything with the **first usable host** for router interfaces and the **last usable** for PCs.
5. From R1, ping every address you configured. (Cross-site LANs won't answer yet — no routes. Say which pings should work now and why; Lab 9 fixes the rest.)

Verification checklist

- Your paper plan: no overlaps, biggest blocks first, every requirement met
- `show ip interface brief` on both routers — everything up/up
- Directly connected pings succeed; cross-site LAN pings fail (for now)

Solution

One clean carve of 172.16.40.0/22 (yours may differ and still be right):

Subnet	Assignment	Range
172.16.40.0/23	HQ staff (510 hosts)	.40.1 – .41.254
172.16.42.0/25	Branch staff (126)	.42.1 – .42.126
172.16.42.128/26	HQ servers (62)	.42.129 – .42.190
172.16.42.192/26	Branch guest (62)	.42.193 – .42.254

Subnet	Assignment	Range
172.16.43.0/30	Router link (2)	.43.1 – .43.2

Everything from 172.16.43.4 up remains free — over a /24 of headroom, which is the point of VLSM: right-sized slices leave room to grow. Guest Wi-Fi needed 40, and the next size that fits is /26 (62), not /27 (30).

Break it

Readdress Branch staff as 172.16.40.128/25 — inside HQ staff's block. Nothing complains immediately (that's the scary part); routing just gets weird for half of HQ. Practice spotting the overlap from `show ip route` on R1.

Lab 9 — Static, Default & Floating Routes

Guide sections: 15.1–15.6 · **Time:** ~40 min

Topology

Three routers in a line, plus a backup path:

Link	Subnet
R1 g0/0 ↔ R2 g0/0	10.0.12.0/30
R2 g0/1 ↔ R3 g0/0	10.0.23.0/30
R1 g0/1 ↔ R3 g0/1	10.0.13.0/30 (the backup)
R1 loopback0	192.168.1.1/24 (pretend LAN)
R3 loopback0	192.168.3.1/24 (pretend LAN)

Tasks

1. Address everything; confirm neighbors ping across each link.
2. Static routes so R1's LAN reaches R3's LAN **via R2** — both directions (remember 15.3: a route one way is only half a conversation). What does R2 need? Reason it out before answering.
3. From R1: `ping 192.168.3.1 source loopback0` — why is sourcing the ping from the loopback the honest test? (What does a plain ping not prove?)
4. Floating statics: add backup routes via the direct R1↔R3 link with AD 90.
5. Prove the float: traceroute from R1 (via R2), shut R2's g0/0, traceroute again (direct), bring it back, watch it return.

6. Replace R1's specific route with a **default** route via R2. When would a real network prefer that (15.4) — and what does R1's routing table lose in precision?

Verification checklist

- `show ip route` — S routes present; the floating one appears **only** while the primary path is down
- Traceroutes before/during/after failure show the path swing

Solution

```
R1(config)# ip route 192.168.3.0 255.255.255.0 10.0.12.2
R3(config)# ip route 192.168.1.0 255.255.255.0 10.0.23.1
R1(config)# ip route 192.168.3.0 255.255.255.0 10.0.13.2 90
R3(config)# ip route 192.168.1.0 255.255.255.0 10.0.13.1 90
```

R2 needs nothing for its directly connected links — but it does need routes to both loopback LANs (`ip route 192.168.1.0 …` / `192.168.3.0 …`)? No — R2 is directly connected to neither LAN but is on the path: it needs a route to each loopback network or it drops the transit packets. Add them and re-test. A plain R1→R3 ping only proves the link subnets work; sourcing from loopback0 forces R3 to know the way back to the LAN — the test users actually experience.

Break it

Delete R3's return route only. R1's pings now fail — but R1's config is perfect. Practice the discipline of 23.1: the fault is not always on the device showing the symptom.

Lab 10 — OSPF Single Area

Guide sections: 16.1–16.11 · **Time:** ~60 min · **Diagram:** the OSPF areas figure in 16.5

Topology

Keep Lab 9's triangle (all three links). Remove **all** static routes (`no ip route …`) — OSPF earns its keep now. Add loopback0 on R2: 192.168.2.1/24.

Tasks

1. Router IDs first: set 1.1.1.1 / 2.2.2.2 / 3.3.3.3 explicitly (16.7 — never let the lab pick for you).

2. Enable OSPF process 1, area 0, on every interface of every router — use `network` statements with exact wildcards (16.8) rather than 0.0.0.0 laziness.
3. Watch `show ip ospf neighbor` until everything is FULL. On the Ethernet links: which router became DR on each segment and why (16.10)?
4. Check `show ip route ospf`: every loopback learned everywhere. What metric did each route get, and can you reproduce the number by hand from 16.6's cost table?
5. Make the R1↔R3 direct link the preferred path R1→R3 by changing **cost** (not bandwidth) — then verify with traceroute.
6. Make the loopbacks advertise their real /24 mask (look at the route table first — what mask do they show and why? `ip ospf network point-to-point` on the loopback is the fix).
7. Kill the preferred link and time the reroute with a continuous ping. Then read `show ip ospf` — how many times has SPF run?
8. Set `passive-interface` on every loopback and explain what it changes and what it deliberately doesn't (16.9's neighbor requirements meet security).

Verification checklist

- `show ip ospf neighbor` — FULL everywhere (2WAY between DROTHERs would only appear with 3+ routers on one segment — explain why if asked!)
- `show ip route ospf` — all loopbacks as /24 after task 6
- Task 5's traceroute takes the direct link

Solution

```
R1(config)# router ospf 1
R1(config-router)# router-id 1.1.1.1
R1(config-router)# network 10.0.12.0 0.0.0.3 area 0
R1(config-router)# network 10.0.13.0 0.0.0.3 area 0
R1(config-router)# network 192.168.1.0 0.0.0.255 area 0
R1(config-router)# passive-interface loopback0
R1(config)# interface loopback0
R1(config-if)# ip ospf network point-to-point
R1(config)# interface g0/1
R1(config-if)# ip ospf cost 5          ! lower than the two-hop path's total
```

(R2/R3 symmetric.) Loopbacks advertise as /32 by default regardless of the configured mask — OSPF treats them as host routes unless told they're point-to-point networks. If you changed router IDs after the neighbors formed, they don't take effect until `clear ip ospf process` — a classic lab head-scratcher.

Break it

Three classics, one per sitting — on one link only: (a) mismatch hello timers (`ip ospf hello-interval 5`), (b) mismatch the area (`area 1` one side), (c) mismatch MTU (`ip mtu 1400` one side). Each produces a different failure signature (no neighbor / stuck in a state / adjacency flaps) — match each to 16.9's table of neighbor requirements.

Lab 11 — IPv6 & SLAAC

Guide sections: 13.1–13.8 · **Time:** ~45 min

Topology

R1 ↔ R2 back-to-back; one VPCS behind each (PC1 on R1's LAN, PC2 on R2's).

Network	Prefix
PC1 LAN	2001:db8:acad:1::/64
R1↔R2 link	2001:db8:acad:12::/64
PC2 LAN	2001:db8:acad:2::/64

Tasks

1. Enable IPv6 routing on both routers (it's off by default — find the command).
2. Address every router interface: `::1` on LAN interfaces, `::1`/`::2` on the link. Watch what else appeared on each interface (`show ipv6 interface`) — identify the link-local address and the multicast groups joined, and match them to 13.4 and 13.8.
3. Let SLAAC do its thing: on PC1 just type `ip auto`. Where did its address come from? Decompose it: whose prefix, and is the host part EUI-64 (13.5)?
4. Static IPv6 routes each way for the far LANs — but use the **link-local** address of the next hop (plus exit interface), the way real IPv6 routing does it. Why must the exit interface be specified with a link-local next hop?
5. PC1 → PC2 ping. Then look at R1's IPv6 neighbor table and find PC1 — this is NDP standing where ARP used to (13.8).

Verification checklist

- `show ipv6 interface brief` — global + link-local on every interface
- PC1's `show ipv6` — SLAAC address inside 2001:db8:acad:1::/64
- `show ipv6 route` — your statics; `show ipv6 neighbors` — PC1's entry

Solution

```
R1(config)# ipv6 unicast-routing
R1(config)# interface g0/0
R1(config-if)# ipv6 address 2001:db8:acad:1::1/64
R1(config-if)# no shutdown
R1(config)# interface g0/1
R1(config-if)# ipv6 address 2001:db8:acad:12::1/64
R1(config-if)# no shutdown
R1(config)# ipv6 route 2001:db8:acad:2::/64 g0/1 fe80::2
```

(R2 mirrored; find its real link-local with `show ipv6 interface g0/1` — `fe80::2` here assumes you set it manually with `ipv6 address fe80::2 link-local`, a common lab nicety.) A link-local next hop exists on every link, so `fe80::…` alone is ambiguous — the exit interface disambiguates. That's also why it's the proper next hop: it never changes when the global prefix gets renumbered.

Break it

Forget `ipv6 unicast-routing` on R2 (or remove it). R2 still answers pings on its own addresses — but stops forwarding and stops sending RAs, so PC2's SLAAC address quietly ages out. Weird, instructive failure.

Lab 12 — HSRP

Guide sections: 14.7 · **Time:** ~40 min · **Diagram:** the HSRP figure in 14.7

Topology

Device	Role
R1, R2	the redundant gateways — g0/0 of each into SW1
SW1 (L2)	the LAN
PC1	VPCS, 192.168.1.10/24, gateway **192.168.1.1**
R1 g0/1, R2 g0/1	each to router R3 ("the internet"), OSPF or statics

R1 g0/0 = 192.168.1.2/24, R2 g0/0 = 192.168.1.3/24. Give R3 a loopback 8.8.8.8/32 to stand in for the outside world.

Tasks

1. Configure HSRP group 1 on the LAN interfaces with virtual IP **192.168.1.1**. Make R1 active by priority, with preemption.
2. From PC1, ping 8.8.8.8 continuously. Note the virtual MAC in PC1's ARP table (`show arp` in VPCS) and decode the group number out of it (14.7).
3. Fail over: shut R1's g0/0. Count lost pings. Check R2 is now active and PC1's ARP entry hasn't changed — the entire trick.
4. Recover R1 and confirm preemption hands the active role back.
5. Bonus if your image supports it: track R1's upstream interface so R1 abdicates when its path to R3 dies, not just when its LAN leg dies — otherwise it keeps the active role while holding a dead uplink.

Verification checklist

- `show standby brief` — R1 Active, R2 Standby, VIP correct
- Task 3: a couple of lost pings, ARP entry unchanged
- Task 4: roles restored automatically

Solution

```
R1(config)# interface g0/0
R1(config-if)# standby 1 ip 192.168.1.1
R1(config-if)# standby 1 priority 110
R1(config-if)# standby 1 preempt
R1(config-if)# standby 1 track g0/1 20     ! bonus: drop 20 below R2 on uplink death
```

R2 the same with default priority (100) and `preempt`. The virtual MAC ends in the group number in hex: `0000.0c07.ac01` for group 1 — recognizing that pattern in an ARP table is a fast exam point and a faster interview point.

Break it

Give R2 `standby 1 ip 192.168.1.5` (wrong VIP). Both routers go Active — each for its own idea of the group. Symptom: intermittent weirdness depending on which router answered PC1's ARP first. `show standby` on both ends exposes the disagreement.

Lab 13 — IP Services: DHCP, NTP, Syslog, SNMP

Guide sections: 17.1, 17.4–17.6 · **Time:** ~50 min · **Diagram:** the DORA figure in 17.1

Topology

Reuse Lab 12's network. R3 becomes the "services core"; PC2 (VPCS) joins SW1's LAN.

Tasks

1. **DHCP server on R1** for 192.168.1.0/24: exclude .1–.9, hand out gateway 192.168.1.1 (the HSRP VIP — a nice touch: leases survive gateway failover) and a DNS address of your choosing. `ip dhcp` on the PCs; watch DORA happen with `debug ip dhcp server events` on R1.
2. **DHCP relay:** create a second LAN on R2 g0/2 (192.168.2.0/24) with PC3 on it, but keep the pool **on R1**. Configure the relay so PC3's broadcast DISCOVER crosses the routed hop, and add a matching pool on R1. Which field of the relayed packet tells R1 which pool to use?

3. **NTP:** R3 as `ntp master 3`; R1 and R2 as clients. Confirm sync and check the stratum R1 reports. What stratum would a device syncing from R1 get?
4. **Syslog:** point R1's logging at PC2's address (nothing will listen, but the config and `show logging` behavior is the exam skill), set console logging to warnings-and-worse, and explain levels 0–7 from memory (17.5).
5. **SNMP:** read-only community `labRO` on R1, restricted by an ACL that permits only PC2's address. Why read-only, and why the ACL (17.6)?

Verification checklist

- `show ip dhcp binding` on R1 — leases for PC1, PC2 and PC3 (two subnets)
- `show ntp status` on R1 — synchronized, stratum 4
- `show logging` — trap/console levels as set

Solution

```
R1(config)# ip dhcp excluded-address 192.168.1.1 192.168.1.9
R1(config)# ip dhcp pool LAN1
R1(dhcp-config)# network 192.168.1.0 255.255.255.0
R1(dhcp-config)# default-router 192.168.1.1
R1(dhcp-config)# dns-server 192.168.1.53
R1(config)# ip dhcp pool LAN2
R1(dhcp-config)# network 192.168.2.0 255.255.255.0
R1(dhcp-config)# default-router 192.168.2.1

R2(config)# interface g0/2
R2(config-if)# ip helper-address 192.168.1.2    ! R1's real address

R1(config)# ntp server <R3 address>
R1(config)# logging host 192.168.1.20
R1(config)# logging console warnings
R1(config)# access-list 10 permit host 192.168.1.20
R1(config)# snmp-server community labRO ro 10
```

The relay stamps the receiving interface's address into **giaddr** — that's how R1 picks the 192.168.2.0 pool for a request that arrived via unicast from another subnet. R1 syncs from stratum 3, so it is stratum 4, and anyone syncing from it lands at 5.

Break it

Remove the helper address. PC3's `ip dhcp` times out silently — no error anywhere on R1, because the DISCOVER broadcast simply never leaves LAN 2. The lesson: DHCP-across-a-router failures live on the router in the middle, and 23.1's layered method finds them fastest.

Lab 14 — NAT & PAT

Guide sections: 17.3 · **Time:** ~40 min · **Diagram:** the PAT figure in 17.3

Topology

Device	Role
R1	the NAT edge — LAN 192.168.1.0/24 inside, 203.0.113.0/30 to R2
R2	"the ISP" — loopback 8.8.8.8/32; a static route back **only** to 203.0.113.0/30
PC1, PC2	VPCS on the LAN

The crucial bit of realism: R2 has **no route** to 192.168.1.0/24 — just like the real internet doesn't (11.4). NAT is what makes the LAN reachable-from anyway.

Tasks

1. Prove the problem first: from PC1, ping 8.8.8.8. Watch it fail and explain where the packet dies — does it even reach R2? (`debug ip icmp` on R2 answers it. The reply is the corpse; find where it fell.)
2. Configure **PAT** on R1: inside/outside interfaces, an ACL matching the LAN, overload on the outside interface.
3. Ping again from both PCs simultaneously and read `show ip nat translations` — find both flows sharing one address, distinguished by port, and name each column with 17.3's four terms (inside local / inside global / outside local / outside global).
4. Add one **static NAT**: map 203.0.113.1's... no — you only have a /30. Map inside 192.168.1.10 to the outside interface's own address for **port 80 only** (port forwarding, the home-router special).
5. `clear ip nat translation *` and explain when a real operator reaches for that command.

Verification checklist

- Task 1's ping fails; after task 2 it works from both PCs
- `show ip nat translations` — two dynamic entries, one static port mapping
- `show ip nat statistics` — hits climbing, correct inside/outside interfaces

Solution

```
R1(config)# interface g0/0
R1(config-if)# ip nat inside
R1(config)# interface g0/1
R1(config-if)# ip nat outside
R1(config)# access-list 1 permit 192.168.1.0 0.0.0.255
R1(config)# ip nat inside source list 1 interface g0/1 overload
R1(config)# ip nat inside source static tcp 192.168.1.10 80 203.0.113.1 80
```

Task 1's autopsy: the request reaches 8.8.8.8 fine (R1 routes it out), but the reply to 192.168.1.10 dies at R2 — no route to private space. Half the value of this lab is having

> personally watched a one-way conversation.

Break it

Reverse `ip nat inside`/`outside` on the two interfaces. Everything is configured, nothing translates, `show ip nat statistics` shows zero hits. A two-word error you'll now spot in seconds forever.

Lab 15 — ACLs

Guide sections: 19.1–19.8 · **Time:** ~50 min · **Diagram:** the ACL flow figure in 19.2

Topology

Reuse Lab 14's network, plus a second LAN on R1 (192.168.2.0/24, PC3) — so traffic between LANs and to "the internet" both cross R1.

Tasks

1. **Standard ACL:** block PC2 (192.168.1.11) from reaching LAN 2 entirely, while PC1 still can. Standard ACLs match only source — so which interface and which direction (19.5's placement rule)? Reason first, place second.
2. Test both the block (PC2→PC3 fails) and the non-block (PC1→PC3 works, PC2→8.8.8.8 still works — did your placement accidentally break that?).
3. **Named extended ACL** `LAN2-POLICY`, applied closest to the source (19.6's placement rule — say why extended gets the opposite rule from standard): permit LAN 2 → anywhere for ICMP and DNS; deny LAN 2 → LAN 1 for everything else; permit the rest.
4. Read your ACL back with `show access-lists` — note the sequence numbers and hit counters. Send test pings and watch the right counter climb; that's how real troubleshooting confirms which line is eating traffic.
5. Protect the vty lines: only PC1's address may SSH to R1 (`access-class` — different command, same idea, and the reason it's separate from interface ACLs is worth a sentence).
6. Insert a rule into the middle of `LAN2-POLICY` using sequence numbers — no delete-and-retype.

Verification checklist

- Ping matrix exactly matches the policy — including the "didn't break" checks
- `show access-lists` — counters incrementing on the lines you predict
- SSH to R1 from PC1 works; from anywhere else refused

Solution

```
R1(config)# access-list 10 deny host 192.168.1.11
R1(config)# access-list 10 permit any
R1(config)# interface g0/2
R1(config-if)# ip access-group 10 out          ! LAN2-facing, outbound

R1(config)# ip access-list extended LAN2-POLICY
R1(config-ext-nacl)# permit icmp 192.168.2.0 0.0.0.255 any
R1(config-ext-nacl)# permit udp 192.168.2.0 0.0.0.255 any eq 53
R1(config-ext-nacl)# deny ip 192.168.2.0 0.0.0.255 192.168.1.0 0.0.0.255
R1(config-ext-nacl)# permit ip any any
R1(config)# interface g0/2
R1(config-if)# ip access-group LAN2-POLICY in

R1(config)# access-list 15 permit host 192.168.1.10
R1(config)# line vty 0 4
R1(config-line)# access-class 15 in
```

Placing standard ACL 10 near the source would also have blocked PC2's internet — matching only on source can't tell destinations apart, so it must sit near the destination. Extended ACLs know both ends, so they filter at the source and spare the network the doomed packet's journey.

Break it

Add `deny ip any any log` as the first line of a fresh ACL "to see what's happening." Everything dies — first match wins, and 19.2's diagram becomes personal experience. Recover via console (your SSH just cut itself off — there's the lesson about editing ACLs on remote devices).

Lab 16 — Layer 2 Defenses

Guide sections: 18.3, 7.5, 8.2 · **Time:** ~45 min

Topology

SW1 (L2) with PC1, PC2; R1 as the legitimate DHCP server on e3/0 (pool for 192.168.1.0/24); and **R4** on e3/1 — the rogue DHCP server you'll configure yourself (same pool, but handing out its own address as gateway: the man-in-the-middle from 18.3).

Tasks

1. Run the attack first: configure R4's rogue pool, then `ip dhcp` on PC1 a few times (release with `clear ip dhcp` between tries). Sometimes the rogue answers first — check which gateway PC1 got. Feel the problem.

2. **DHCP snooping** on SW1: enable globally and for VLAN 1, mark only R1's port trusted, and rate-limit untrusted ports. Repeat the attack: the rogue OFFER now dies at the switch.
3. Inspect the snooping **binding table** — PC1's lease is recorded there. That table is the fuel for the next defense.
4. **Dynamic ARP Inspection** for VLAN 1, R1's port trusted. From PC2, try to ARP-poison... VPCS can't forge ARP, so simulate: give PC2 a static IP that doesn't match any DHCP binding and watch DAI drop its ARP (check the log and `show ip arp inspection statistics`). What legitimate scenario just broke, and how does a real network handle statically-addressed hosts with DAI on?
5. Finish the hygiene checklist from Chapters 7–8 on SW1: shut unused ports, park them in an unused VLAN, native VLAN off 1, no DTP on trunks.

Verification checklist

- Attack works before task 2, fails after — `show ip dhcp snooping` confirms trusted ports
- `show ip dhcp snooping binding` — PC1's entry
- DAI statistics show drops for the unbound host

Solution

```
SW1(config)# ip dhcp snooping
SW1(config)# ip dhcp snooping vlan 1
SW1(config)# no ip dhcp snooping information option   ! IOS quirk: needed for a router server
SW1(config)# interface e3/0
SW1(config-if)# ip dhcp snooping trust
SW1(config)# interface range e0/1 - 2
SW1(config-if-range)# ip dhcp snooping limit rate 10

SW1(config)# ip arp inspection vlan 1
SW1(config)# interface e3/0
SW1(config-if)# ip arp inspection trust
```

Task 4's casualty: any host with a static IP has no DHCP binding, so DAI drops its ARP — real networks add **ARP ACLs** for static hosts (or trust their ports, reluctantly). The `information option` line is a common lab gotcha: snooping inserts option 82 by default and many IOS DHCP servers then ignore the request.

Break it

Forget to trust R1's port. DHCP dies for everyone — snooping eats the legitimate OFFERs too. Diagnose from PC1's timeout back through `show ip dhcp snooping` — the fastest tell is DHCP working before the feature and dying after.

Lab 17 — Capstone: Build a Company

Guide sections: all of them · **Time:** a weekend, honestly

No hand-holding now. Build **AcmeCo**: two sites, real requirements, your design decisions. When it works, break-and-fix challenges close the book.

The brief

HQ: two L2 access switches, one L3 core switch, two edge routers (R1, R2). **Branch:** one router (R3), one switch. **"Internet":** R5 with loopback 8.8.8.8, reachable only from R1/R2's outside addresses (like Lab 14).

Requirement	Details
VLANs at HQ	10 Staff, 20 Servers, 30 Voice, 99 native/management — trunks between all HQ switches, EtherChannel between core and each access switch
Addressing	Carve everything from **10.10.0.0/16** with VLSM; document it before configuring
Routing	OSPF area 0 everywhere internal; default route to R5 originated by the edge
Gateway redundancy	HSRP at HQ for VLAN 10, R1 primary, preemption, uplink tracking
Services	DHCP for Staff + Branch (server on the core or R1, relay where needed), NTP from R5 down, syslog levels set everywhere
Internet access	PAT at the edge; one port-forward to a Server-VLAN host
Security	SSH-only management, port security on access ports, DHCP snooping + DAI at HQ access, ACL: Staff may not reach Servers except DNS+HTTP, guest rules of your choosing
STP	Core switch is root, PortFast+BPDU Guard on all host ports, root guard toward access

Grade yourself

- A PC in HQ Staff gets a DHCP lease, resolves through the allowed ports to a Server-VLAN host, reaches 8.8.8.8, and survives: R1's LAN interface dying, R1's uplink dying, and any single inter-switch link dying — with at most a few lost pings each.
- A Branch PC gets its lease across the relay and reaches both HQ servers and the internet.

- Every "may not" in the security table provably fails, and every "may" provably works — build the ping/SSH matrix and check it off line by line.
- `show` evidence exists for each requirement. If you can't point at the command that proves a row, it isn't done.

Break-and-fix finale

Export the working configs first. Then, one at a time, self-inflict each of these, observe symptoms from a user's point of view, and fix without looking at what you changed (or better — have someone else break it):

1. Native VLAN mismatch on one HQ trunk
2. Wrong `encapsulation dot1q` number on one subinterface or SVI VLAN
3. OSPF hello-timer mismatch on the HQ↔Branch link
4. `ip nat inside`/`outside` reversed
5. An ACL line added at the top instead of by sequence number
6. HSRP VIPs disagreeing between R1 and R2
7. DHCP snooping trust removed from the server port
8. The config register left at 0x2142 on R3 (then recover via 21.5's walk)

When all eight feel routine, you are — in the most literal sense — ready for the troubleshooting section of the exam, and for the interview whiteboard.

Where Labs Can't Take You (Yet)

Two blueprint areas resist EVE-NG, and it's better to say so than pretend:

- **Wireless (WLC GUI):** IOL/vIOS have no radios and the virtual WLC image is heavyweight and licensed. The exam shows you **screenshots** of the WLC GUI (20.10's workflow) — practice that flow in **Cisco Packet Tracer** (its WLC object is simplified but walks the same WLAN → interface → security → QoS screens), or in the Cisco DevNet sandbox if you have an account.
- **Automation APIs:** the REST/JSON topics (22.4–22.5) are read-and-interpret on the exam, not configure. If you want them hands-on anyway: the DevNet **Always-On sandboxes** expose real device APIs to poke with `curl` or Postman — entirely optional for the exam, quietly impressive in interviews.

Everything else in the blueprint, you have now typed. That's the difference between knowing the answer and being the person they hire.

— End of Lab Workbook —